



NOUVELLE AVEIRO MAROC
Industrie & Maintenance



ROYAUME DU MAROC
FORMATION PROFESSIONNELLE
INSTITUT SPÉCIALISÉ DE TECHNOLOGIE
APPLIQUÉE AÏT MELLOUL

Rapport de stage

Développement d'une Application Web de Gestion de Maintenance Industrielle — FixFlow

Stage effectué au sein de l'entreprise NOUVELLE AVEIRO MAROC

du 01 avril 2026 au 30 avril 2026

Dans le cadre de la formation de Technicien Spécialisé

SPÉCIALITÉ : DÉVELOPPEMENT DIGITAL
OPTION : FULL STACK

Rédigé par :
BENCHOUAIB CHOUAIB

Tuteur de stage :
M. ACHRAF KARMAOUI

Encadrée par :
Mme KHADIJA BACHTCHI



DEDICACE

Je dédie ce travail :

À mes chers parents,

Pour leur amour inconditionnel, leurs sacrifices et leur soutien tout au long de mon parcours. Vos prières et encouragements ont été ma force et ma motivation. Ce travail est le fruit de votre confiance en moi.

À mes frères,

Pour les moments de joie partagés, votre patience et votre compréhension durant cette période de formation.

À mon tuteur de stage, M. Achraf Karmaoui,

Pour votre accueil chaleureux, votre disponibilité et votre précieux encadrement. Votre expertise et vos conseils ont été déterminants dans la réussite de ce projet.

À ma formatrice, Mme Khadija Bachtchi,

Pour la qualité de votre enseignement, votre pédagogie bienveillante et votre soutien constant tout au long de ces deux années de formation.

À tous mes amis et camarades de promotion,

Pour les moments de partage, d'entraide et de convivialité qui ont enrichi mon parcours académique.

Je vous dédie ce travail en témoignage de ma profonde gratitude et de mon affection.



REMERCIEMENTS

Avant tout, je tiens à exprimer ma profonde gratitude à M. ACHRAF KARMAOUI, mon encadrant au sein de Nouvelle Aveiro Maroc, pour m'avoir accueilli avec bienveillance et guidé tout au long de ce stage avec patience et professionnalisme. Sa disponibilité et ses précieux conseils ont été déterminants dans la réussite de cette expérience.

Je remercie chaleureusement toute l'équipe de Nouvelle Aveiro Maroc pour leur accueil, leur esprit d'entraide et l'atmosphère de travail agréable qu'ils ont su créer. Travailler à leurs côtés a été une expérience humaine et professionnelle vraiment enrichissante.

Mes sincères remerciements vont également à Mme KHADIJA BACHTCHI, ma formatrice à l'ISTA Aït Melloul, pour la qualité de l'enseignement qu'elle m'a transmis tout au long de ces deux années, sa pédagogie bienveillante et son soutien constant qui m'ont permis d'aborder ce stage avec confiance et sérénité.

Enfin, je souhaite dédier ces remerciements à ma famille, dont le soutien indéfectible, la confiance et les encouragements au quotidien m'ont donné la force et la motivation nécessaires pour mener ce projet à bien. Ce rapport leur appartient autant qu'à moi.



RESUME

Ce rapport présente le travail réalisé dans le cadre d'un stage de fin de formation effectué au sein de Nouvelle Aveiro Maroc, du 01 avril 2026 au 30 avril 2026, dans le cadre de la filière Développement Digital Option Full Stack à l'ISTA Aït Melloul.

La mission principale de ce stage consistait à concevoir et développer FixFlow, une application web de gestion de maintenance industrielle. En l'absence d'un outil centralisé, les opérateurs ne disposaient d'aucun moyen structuré pour signaler les pannes machines, et les techniciens ne pouvaient pas suivre efficacement les interventions.

L'application développée permet aux opérateurs de signaler des pannes via des tickets (avec scan QR des machines), aux techniciens de les traiter et mettre à jour leur statut en temps réel, et aux administrateurs de superviser l'ensemble du système via un dashboard analytique.

Sur le plan technique, le projet repose sur une architecture client-serveur : backend API REST avec Laravel 13 (PHP 8.4+), Laravel Sanctum pour l'authentification, SQLite/MySQL pour la base de données, et frontend SPA avec React 19 + Vite 8, Tailwind CSS 4, React Router DOM 7 et Axios.

Cette expérience m'a permis d'approfondir mes compétences en développement full stack, en architecture logicielle et en gestion de projet, tout en travaillant dans un environnement industriel professionnel et stimulant.

Mots-clés : Laravel 13 · React 19 · API REST · Sanctum · SQLite · Gestion de maintenance · Tickets · QR Code · Full Stack



GLOSSAIRE

API REST : Application Programming Interface Representational State Transfer — Interface permettant

la communication entre le frontend et le backend via des requêtes HTTP standardisées.

Axios : Bibliothèque JavaScript permettant d'effectuer des requêtes HTTP depuis le frontend React vers

l'API backend.

Context API : Mécanisme React permettant de partager un état global (auth, tickets, machines, langue)

entre les composants sans prop drilling.

CRUD : Create, Read, Update, Delete — Les quatre opérations de base pour la gestion des données.

Eloquent ORM : ORM intégré à Laravel permettant d'interagir avec la base de données via des modèles

PHP orientés objet.

FixFlow : Nom du projet développé — application web de gestion de maintenance industrielle.

html5-qrcode : Bibliothèque JavaScript permettant le scan de QR codes via la caméra du navigateur pour

identifier les machines.

Laravel Sanctum : Package Laravel fournissant un système d'authentification léger par token API pour

les SPA.

Lucide-react : Bibliothèque d'icônes React utilisée pour l'interface utilisateur de FixFlow.

Migration : Script Laravel définissant la structure des tables de base de données de manière versionnée.

MVC : Model-View-Controller — Pattern architectural séparant les données, l'interface utilisateur et la

logique de contrôle.

React Router DOM : Bibliothèque de routage côté client pour les applications React Single Page



Application.

Seeder : Script Laravel permettant de peupler la base de données avec des données de test initiales.

SPA : Single Page Application — Application web dont le contenu est chargé dynamiquement sans rechargement complet de la page.

Tailwind CSS : Framework CSS utilitaire permettant de construire des interfaces modernes rapidement.

Ticket : Entité centrale de FixFlow représentant un signalement de panne ou une demande de maintenance sur une machine.

Vite : Outil de build ultra-rapide utilisé pour le développement et la compilation du frontend React.



LISTE DES FIGURES

Chapitre 1 — Contexte général du projet

Figure 1.1	Logo de Nouvelle Aveiro Maroc	8
Figure 1.2	Organigramme de Nouvelle Aveiro Maroc	9

Chapitre 2 — Analyse et conception du projet

Figure 2.1	Diagramme de cas d'utilisation global	15
Figure 2.2	Diagramme de cas d'utilisation — Module Tickets	16
Figure 2.3	Diagramme de cas d'utilisation — Module Machines	17
Figure 2.4	Diagramme de classes	19
Figure 2.5	Diagramme de séquence — Création d'un ticket	20
Figure 2.6	Diagramme de séquence — Traitement d'une panne	21
Figure 2.7	Diagramme d'activité — Processus de gestion de ticket	22
Figure 2.8	Diagramme d'état — Cycle de vie d'un ticket	23
Figure 2.9	Modèle Conceptuel de Données (MCD)	24
Figure 2.10	Modèle Logique de Données (MLD)	26
Figure 2.11	Architecture technique client-serveur	28

Chapitre 3 — Réalisation et mise en œuvre

Figure 3.1	Interface de connexion (Login)	32
Figure 3.2	Dashboard administrateur avec KPI	33
Figure 3.3	Interface opérateur — Signalement de panne	35
Figure 3.4	Interface technicien — Gestion des tickets	36
Figure 3.5	Scan QR Code machine	37



Figure 3.6	Vue liste des machines	38
Figure 3.7	Formulaire de création de ticket	39
Figure 3.8	Gestion des utilisateurs et rôles	40
Figure 3.9	Journal des logs système	41
Figure 3.10	Structure du projet FixFlow	42



LISTE DES TABLEAUX

Chapitre 1 — Contexte général du projet

Tableau 1.1	Informations générales du projet	12
Tableau 1.2	Matrice des droits d'accès par profil utilisateur	13

Chapitre 2 — Analyse et conception du projet

Tableau 2.1	Ordre de création des migrations	17
Tableau 2.2	Structure de la table machines	18
Tableau 2.3	Structure de la table tickets	19
Tableau 2.4	Structure de la table logs	20
Tableau 2.5	Tables référentielles utilisateurs	21
Tableau 2.6	Stack technologique du projet	27

Chapitre 3 — Réalisation et mise en œuvre

Tableau 3.1	Outils de développement utilisés	31
Tableau 3.2	Répartition des tâches par phase	46
Tableau 3.3	Statuts des tickets et leur signification	47
Tableau 3.4	Rôles et permissions utilisateurs	49
Tableau 3.5	Checklist de déploiement en production	51



TABLE DES MATIERES

DEDICACE	2
REMERCIEMENTS	3
RESUME	4
GLOSSAIRE	5
LISTE DES FIGURES	7
LISTE DES TABLEAUX	8
TABLE DES MATIERES	9
INTRODUCTION GENERALE	12
CHAPITRE 1 : CONTEXTE GÉNÉRAL DU PROJET	14
I. Présentation de l'organisme d'accueil	14
II. Les tâches effectuées	16
Analyse et conception	16
Développement backend	16
Développement frontend	16
Gestion de projet et documentation	17
III. Présentation de l'existant et problématique	18
Analyse de l'existant	18
Problèmes identifiés	18
Impact sur l'activité	19
IV. Solution proposée et objectifs du projet	20



Description de la solution FixFlow	20
Objectifs du projet	20
Périmètre du projet	21
Informations générales du projet	21
V. Matrice des droits d'accès	22
 CHAPITRE 2 : ANALYSE ET CONCEPTION DU PROJET	 23
I. Les besoins fonctionnels	23
Module Gestion des Tickets	23
Module Gestion des Machines	23
Module Tableau de bord	24
Module Administration	24
Diagramme de cas d'utilisation	24
II. Choix de la méthode de conception	27
III. Dictionnaire de données	28
IV. Modèle Conceptuel de Données (MCD)	31
V. Modèle Logique de Données (MLD)	33
VI. Architecture technique	37
VII. Diagrammes UML	41
 CHAPITRE 3 : RÉALISATION ET MISE EN ŒUVRE	 47
I. Les outils de travail	47
II. Environnement de développement	49
III. Structure du projet	50
IV. Les principales interfaces	55



V. Tests et validation	75
VI. Déploiement	77
CONCLUSION	80
RÉFÉRENCES	82



INTRODUCTION GENERALE

Dans le cadre de la formation de Technicien Spécialisé en Développement Digital option Full Stack à l'Institut Spécialisé de Technologie Appliquée (ISTA) d'Aït Melloul, un stage de fin de formation est obligatoire pour valider les compétences acquises au cours des deux années de formation. Ce stage, d'une durée d'un mois, constitue une passerelle essentielle entre le monde académique et le monde professionnel.

J'ai eu l'opportunité d'effectuer mon stage au sein de Nouvelle Aveiro Maroc, une entreprise industrielle implantée à Aït Melloul. Cette entreprise, spécialisée dans la production industrielle, fait face à un défi majeur : l'absence d'un outil centralisé permettant de gérer efficacement les pannes machines et les interventions de maintenance.

La mission qui m'a été confiée consistait à concevoir et développer FixFlow, une application web complète de gestion de maintenance industrielle. Cette application devait permettre aux opérateurs de signaler les pannes via des tickets, aux techniciens de les traiter et de suivre les interventions, et aux administrateurs de superviser l'ensemble du système via un dashboard analytique. L'application devait répondre aux besoins spécifiques de trois profils d'utilisateurs distincts : opérateur (Worker), technicien et administrateur.

Ce rapport de stage présente de manière structurée l'ensemble du travail réalisé. Il est organisé en trois chapitres principaux :

Le premier chapitre, « Contexte Général du Projet », présente l'organisme d'accueil Nouvelle Aveiro Maroc, les missions confiées durant le stage, l'analyse de l'existant et les problématiques identifiées, ainsi que la solution proposée et les objectifs du projet.

Le deuxième chapitre, « Analyse et Conception du Projet », détaille les besoins fonctionnels identifiés, le dictionnaire de données, les modèles de données, l'architecture technique retenue, et les diagrammes UML.

Le troisième chapitre, « Réalisation et Mise en Œuvre », présente les outils et technologies utilisés, la structure du projet, les principales interfaces de l'application développée, les tests effectués, et enfin le processus de déploiement.

Ce rapport se conclura par une synthèse du travail accompli, les perspectives d'évolution du



projet, ainsi que l'apport personnel et professionnel de cette expérience dans mon cursus de formation.



CHAPITRE 1 : CONTEXTE GÉNÉRAL DU PROJET

I. Présentation de l'organisme d'accueil :

Nouvelle Aveiro Maroc est une entreprise industrielle implantée à Aït Melloul, dans la région Souss-Massa. Spécialisée dans la production industrielle, l'entreprise dispose d'un parc machines important dont la disponibilité et la fiabilité sont essentielles à la continuité de la production.

L'activité principale de Nouvelle Aveiro Maroc repose sur trois piliers fondamentaux :

- La production industrielle : Fabrication de produits selon des processus rigoureux nécessitant des machines en parfait état de fonctionnement.
- La maintenance préventive et corrective : Entretien régulier et intervention rapide en cas de panne pour minimiser les arrêts de production.
- Le support technique : Assistance technique pour garantir la performance et la durée de vie des équipements industriels.

Les machines de Nouvelle Aveiro Maroc constituent une ressource critique pour l'entreprise. La gestion optimale de leur maintenance conditionne directement la productivité et la qualité de la production.

L'entreprise emploie une équipe multidisciplinaire comprenant des opérateurs de production, des techniciens de maintenance, des responsables de production et une direction stratégique.

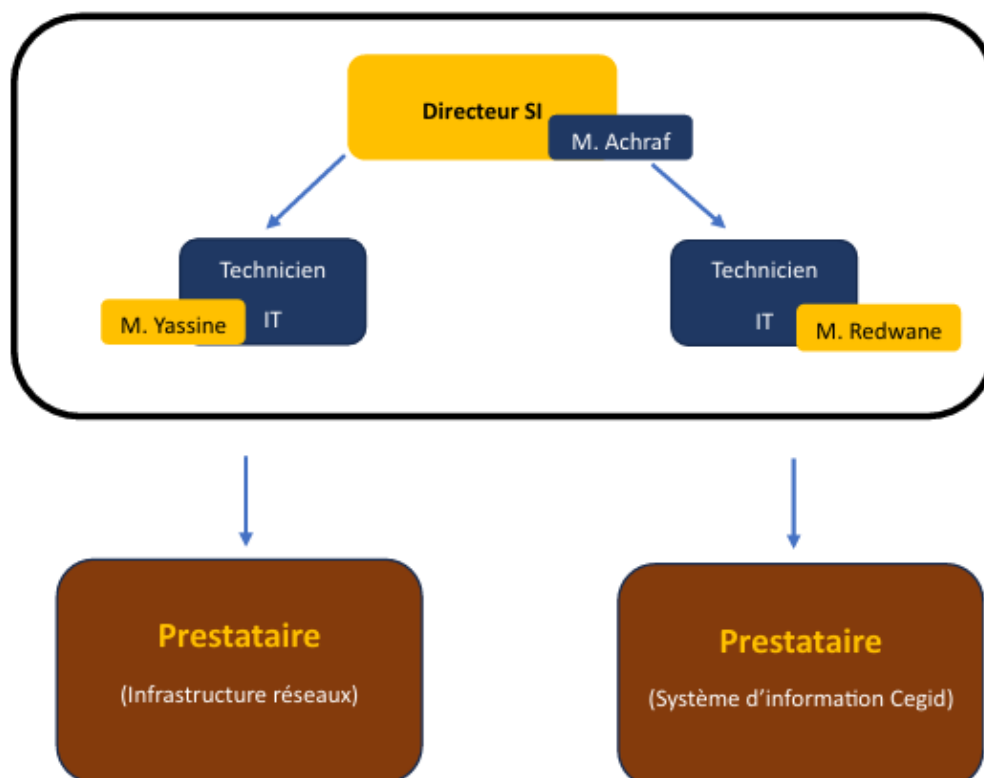
L'organigramme hiérarchique est structuré comme suit :

- Direction Générale : Définit la stratégie globale et prend les décisions majeures.
- Responsable Maintenance : Supervise les interventions techniques et les techniciens.
- Techniciens : Assurent la maintenance et les réparations des machines.
- Opérateurs : Utilisent les machines et signalent les anomalies.
- Équipe Support : Accompagne les équipes dans leurs activités quotidiennes.



[Figure 1.1 : Logo de Nouvelle Aveiro Maroc]

➤ Organigramme technique département Informatique



[Figure 1.2 : Organigramme de Nouvelle Aveiro Maroc]

II. Les tâches effectuées :

Durant mon stage au sein de Nouvelle Aveiro Maroc, j'ai été intégré au pôle technique. Sous la supervision de M. Achraf Karmaoui, mon tuteur de stage, j'ai eu la responsabilité de mener à bien plusieurs missions essentielles. Ces tâches peuvent être regroupées selon les thématiques suivantes :

Analyse et conception :

- Étude approfondie de l'existant et identification des problématiques liées à la gestion des pannes machines.
- Recueil des besoins auprès des différents intervenants (direction, responsable maintenance, opérateurs, techniciens).
- Rédaction du cahier des charges complet définissant les fonctionnalités, les contraintes techniques et les livrables.
- Conception de l'architecture technique client-serveur et définition du modèle de données (4 tables principales).
- Élaboration des diagrammes UML (cas d'utilisation, classes, séquences, activités, états).

Développement backend :

- Initialisation du projet Laravel 13 avec la structure complète (Controllers, Models, Migrations, Seeders).
- Création des migrations : users, machines, tickets, logs.
- Développement de l'API REST avec les contrôleurs : AuthController, TicketController, MachineController, UserController.
- Implémentation de l'authentification par token avec Laravel Sanctum.
- Développement du système de journalisation des actions (Log model).
- Configuration des routes API dans routes/api.php avec gestion des middlewares et des rôles.

Développement frontend :

- Initialisation du projet React 19 + Vite 8 avec Tailwind CSS 4.
- Développement des trois espaces utilisateurs : pages/worker/, pages/technician/, pages/admin/.



-
- Implémentation des contextes globaux : AuthContext, TicketContext, MachineContext, LanguageContext.
 - Intégration du scan QR Code via html5-qrcode pour l'identification des machines.
 - Développement de 72+ composants UI dont TicketForm, QRScanner, MachineCard.
 - Mise en place du routage avec React Router DOM 7 et des layouts par profil.

Gestion de projet et documentation :

- Organisation et suivi des tâches par phase de développement.
- Rédaction de la documentation technique (README, guide d'installation, guide API).
- Rédaction de la documentation utilisateur par profil (admin, technicien, opérateur).
- Préparation du rapport de recette avec la checklist des livrables.
-

III. Présentation de l'existant et problématique :

Avant le début de ce projet, Nouvelle Aveiro Maroc ne disposait d'aucun outil centralisé permettant de suivre et de gérer les pannes machines. Les signalements étaient effectués de manière informelle (oral, papier, messages), sans traçabilité ni suivi structuré.

Analyse de l'existant

- Les pannes machines étaient signalées verbalement ou par messages sans format standardisé.
- Les techniciens ne disposaient d'aucune vue centralisée des interventions à effectuer.
- La direction n'avait aucun moyen de mesurer les taux de pannes, les temps d'intervention ou les machines les plus défectueuses.
- L'identification des machines était manuelle et sujette aux erreurs.
- Aucun historique des interventions n'était conservé de manière structurée.

Problèmes identifiés

- Absence de suivi en temps réel des pannes et interventions : Impossible de connaître l'état des tickets en cours.
- Pas de traçabilité des interventions : Les réparations effectuées n'étaient pas documentées.
- Aucun indicateur de performance : Pas de KPI sur les temps d'arrêt, les taux de résolution, les pannes récurrentes.
- Identification manuelle des machines : Risque de confusion entre machines similaires.
- Communication inefficace : Entre opérateurs et techniciens, sans canal dédié et structuré.

Impact sur l'activité

- Allongement des temps d'arrêt machines faute de réactivité dans la prise en charge des pannes.
- Perte de productivité due aux pannes non détectées à temps ou mal gérées.
- Difficulté à planifier la maintenance préventive sans historique des incidents.
- Impossibilité de mesurer le retour sur investissement de l'équipe de maintenance.

IV. Solution proposée et objectifs du projet :

Description de la solution :

FixFlow est une application web de gestion de maintenance industrielle développée avec une architecture client-serveur. Le backend est une API REST construite avec Laravel 13, sécurisée par Laravel Sanctum. Le frontend est une SPA (Single Page Application) développée avec React 19 + Vite 8. L'application distingue trois profils d'utilisateurs avec des espaces dédiés et des permissions granulaires.

- Couche 1 - Frontend SPA React : Interface utilisateur adaptée à chaque profil (Worker, Technicien, Admin).

- Couche 2 - API REST Laravel : Gestion des requêtes, authentification Sanctum, logique métier.
- Couche 3 - Base de données SQLite/MySQL : Persistance des données (users, machines, tickets, logs).
-

Objectifs du projet :

- Permettre aux opérateurs de signaler les pannes machines via des tickets structurés avec scan QR.
- Permettre aux techniciens de visualiser, prendre en charge et mettre à jour les tickets d'intervention.
- Fournir aux administrateurs un dashboard avec KPI (nombre de tickets, taux de résolution, machines critiques).
- Identifier les machines rapidement et sans erreur via scan de QR Code.
- Conserver un historique complet de toutes les interventions et actions.
- Assurer la sécurité via un système d'authentification par token (Sanctum).

Périmètre du projet

Dans le périmètre :

- Signalement de pannes via tickets
- Gestion du cycle de vie des tickets (création → en cours → résolu)
- Scan QR Code pour identification des machines
- Dashboard analytique admin
- Gestion des utilisateurs et des rôles
- Journal des logs d'actions

Hors périmètre :

- Gestion de la production ou de la planification

- Application mobile native
- Intégration avec des ERP ou outils tiers
- Maintenance prédictive par IA

Tableau 1.1 : Informations générales du projet

Champ	Valeur
Projet	Application FixFlow — Gestion de Maintenance Industrielle
Client	Nouvelle Aveiro Maroc
Version	1.0 – Finale
Date	Avril 2026
Technologie	Laravel 13 · React 19 · Vite 8 · Tailwind CSS 4 · SQLite/MySQL · Sanctum
Équipe	1 développeur full-stack
Durée totale	4 semaines · 5 phases
Nombre de tables DB	4 tables principales (users, machines, tickets, logs)

V. Matrice des droits d'accès :

Fonctionnalité	Admin	Technicien	Opérateur
Dashboard global	Oui	Non	Non
Créer un ticket	Oui	Non	Oui
Traiter un ticket	Oui	Oui	Non
Voir ses tickets	Oui	Oui	Oui
Scanner QR machine	Oui	Oui	Oui



Gérer les machines	Oui	Non	Non
Gérer les utilisateurs	Oui	Non	Non
Voir les logs	Oui	Non	Non

Tableau 1.2 : Matrice des droits d'accès par profil utilisateur

CHAPITRE 2 : ANALYSE ET CONCEPTION DU PROJET

I. Les besoins fonctionnels :

L'analyse des besoins a été réalisée en collaboration avec les différents intervenants de Nouvelle Aveiro Maroc. Elle a permis d'identifier les besoins fonctionnels essentiels que l'application FixFlow doit satisfaire.

Module Gestion des Tickets

- Créer un ticket de panne avec description, priorité et machine concernée.
- Scanner le QR Code d'une machine pour l'identifier automatiquement lors de la création du ticket.
- Mettre à jour le statut d'un ticket : ouvert → en cours → résolu → fermé.
- Affecter un ticket à un technicien spécifique.
- Consulter la liste de ses propres tickets (pour l'opérateur).
- Consulter tous les tickets en attente ou en cours (pour le technicien).

Module Gestion des Machines

- Lister toutes les machines avec leurs informations (nom, code, localisation, état).
- Créer, modifier et désactiver une machine (admin uniquement).
- Générer un QR Code unique pour chaque machine.
- Consulter l'historique des tickets par machine.

Module Tableau de bord (Admin)

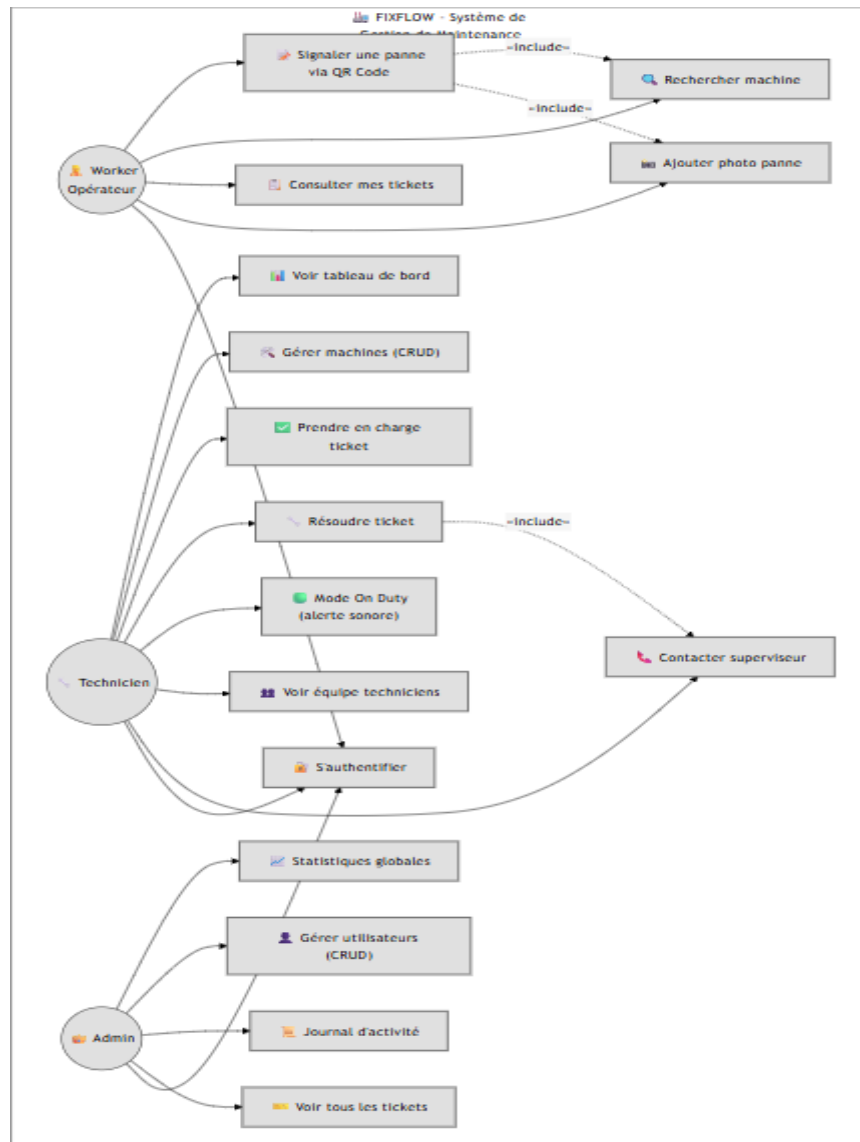
- Afficher les KPI principaux : nombre total de tickets, tickets ouverts, tickets résolus, taux de résolution.



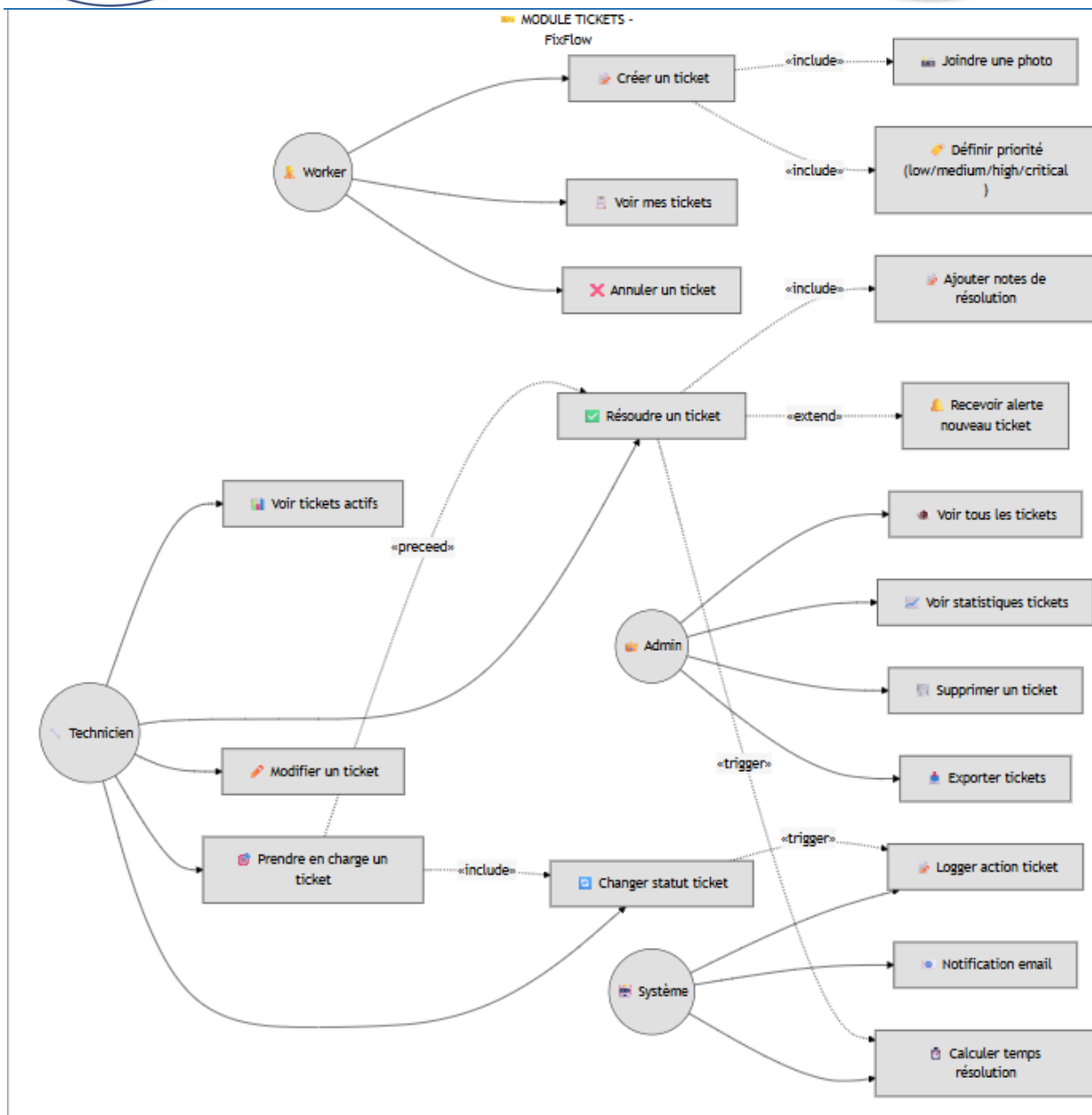
-
- Visualiser la répartition des tickets par statut et par machine.
 - Identifier les machines les plus défectueuses.
 - Consulter les interventions récentes.

Module Administration

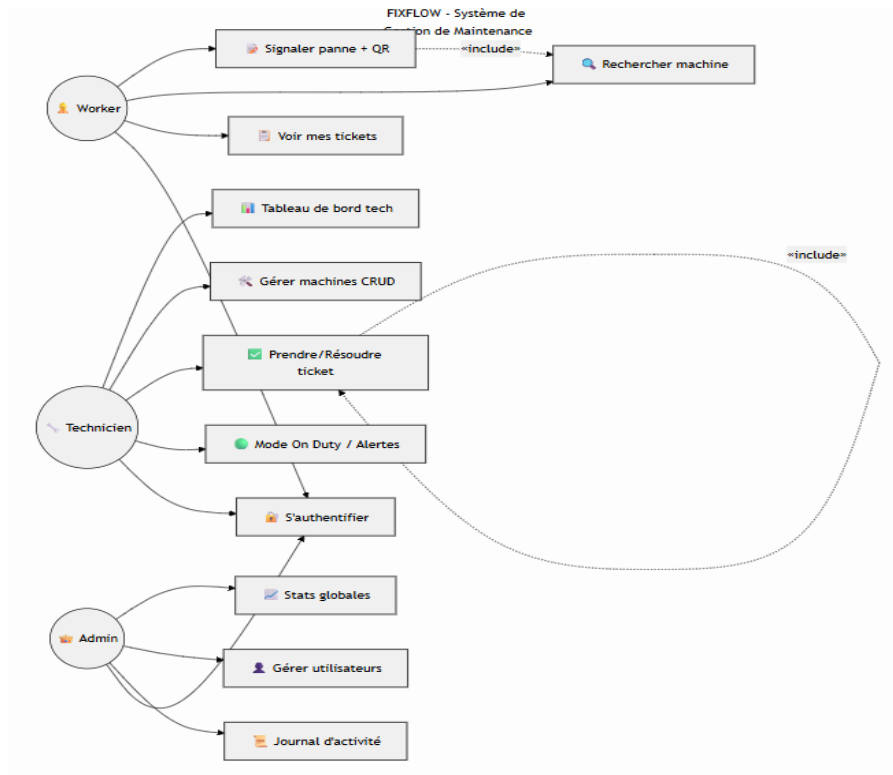
- Gérer les utilisateurs (créer, modifier, désactiver) avec attribution de rôle (admin, technicien, opérateur).
- Consulter les logs de toutes les actions effectuées dans le système.
- Configurer les paramètres généraux de l'application.



[Figure 2.1 : Diagramme de cas d'utilisation global]



[Figure 2.2 : Diagramme de cas d'utilisation — Module Tickets]



[Figure 2.3 : Diagramme de cas d'utilisation — Module Machines]

II. Choix de la méthode de conception :

Pour la conception et le développement de FixFlow, une approche pragmatique a été adoptée, combinant les principes du MVC (Model-View-Controller) avec une méthodologie de développement itérative. Ce choix a été motivé par la durée limitée du stage (4 semaines) et la nécessité de livrer rapidement une version fonctionnelle.

L'architecture client-serveur choisie sépare clairement les responsabilités :

- Backend Laravel (API REST) : Gestion des données, logique métier, authentification, sécurité.
- Frontend React (SPA) : Interface utilisateur, interactions, navigation, appels API via Axios.

Le développement a été organisé en 5 phases itératives :

- Phase 1 : Setup du projet, configuration Laravel + React, base de données, authentification Sanctum.
- Phase 2 : API tickets et machines, CRUD complet, middleware de rôles.
- Phase 3 : Frontend — espaces Worker, Technicien, Admin avec leurs fonctionnalités de base.
- Phase 4 : Intégration QR Code, dashboard KPI, logs, notifications.
- Phase 5 : Tests, corrections, documentation, déploiement.

III. Dictionnaire de données :

Le dictionnaire de données présente l'ensemble des tables, attributs, types et contraintes définis pour la base de données de FixFlow.

Ordre de création des migrations



Ordre	Table	Catégorie	Dépend de
1	users	Authentification	Aucune (Sanctum)
2	machines	Core	Aucune
3	tickets	Core	users, machines
4	logs	Système	users, tickets

Tableau 2.1 : Ordre de création des migrations

Table machines (Core)

Attribut	Type	Contrainte	Description
id	bigint unsigned	PK · auto-increment	Identifiant unique
name	varchar(150)	required	Nom de la machine
code	varchar(50)	unique · required	Code QR unique
location	varchar(100)	nullable	Emplacement physique
description	text	nullable	Description libre
status	enum	active/inactive	État de la machine
created_at / updated_at	timestamp	auto	Gérés par Laravel

Tableau 2.2 : Structure de la table machines

Table tickets (Core — table centrale)

Attribut	Type	Contrainte	Description
id	bigint unsigned	PK · auto-increment	Identifiant unique
machine_id	bigint unsigned	FK → machines.id	Machine concernée
created_by	bigint unsigned	FK → users.id	Opérateur ayant signalé
assigned_to	bigint unsigned	FK · nullable	Technicien assigné
title	varchar(255)	required	Titre du ticket
description	text	nullable	Description de la panne
priority	enum	low/medium/high/urgent	Niveau de priorité
status	enum	open/in_progress/resolved/closed	État courant du ticket
resolved_at	timestamp	nullable	Date de résolution
created_at / updated_at	timestamp	auto	Gérés par Laravel

Tableau 2.3 : Structure de la table tickets

IV. Modèle Conceptuel de Données (MCD) :

Le MCD représente les entités du système et leurs associations. Les entités principales de FixFlow sont USER, MACHINE, TICKET et LOG.

Associations entre entités

- USER (1) — (*) TICKET (created_by) : Un utilisateur peut créer plusieurs tickets.
- USER (1) — (*) TICKET (assigned_to) : Un technicien peut être assigné à plusieurs tickets.
- MACHINE (1) — (*) TICKET : Une machine peut avoir plusieurs tickets de panne.
- USER (1) — (*) LOG : Un utilisateur génère plusieurs entrées de log.
- TICKET (1) — (*) LOG : Un ticket peut générer plusieurs entrées de log.

V. Modèle Logique de Données (MLD)

Schéma relationnel complet

- users (id, name, email, password, role, is_active, remember_token, created_at, updated_at)
- machines (id, name, code, location, description, status, created_at, updated_at)
- tickets (id, machine_id, created_by, assigned_to, title, description, priority, status, resolved_at, created_at, updated_at)
- logs (id, user_id, ticket_id, action, details, created_at, updated_at)

Contraintes d'intégrité référentielle

- tickets.machine_id → machines.id (ON DELETE CASCADE)
- tickets.created_by → users.id (ON DELETE CASCADE)
- tickets.assigned_to → users.id (ON DELETE SET NULL)
- logs.user_id → users.id (ON DELETE CASCADE)
- logs.ticket_id → tickets.id (ON DELETE SET NULL)
-

VI. Architecture technique

Stack technologique

Composant	Technologie	Version	Rôle
Framework backend	Laravel	13.x	MVC, routing, API REST, queues
Langage	PHP	8.4+	Langage serveur principal
Base de données	SQLite / MySQL	3.x / 8.0	Persistance des données
Auth	Laravel Sanctum	4.x	Authentification par token API
ORM	Eloquent ORM	—	Accès base de données
Framework frontend	React	19.x	SPA, composants UI
Bundler	Vite	8.x	Build ultra-rapide du frontend
CSS	Tailwind CSS	4.x	Styles utilitaires
Routage	React Router DOM	7.x	Navigation SPA côté client
HTTP Client	Axios	1.x	Requêtes API depuis le frontend



QR Code	html5-qrcode	2.x	Scan QR des machines
Icônes	Lucide-react	0.x	Interface utilisateur
Animations	Motion (Framer)	—	Animations UI fluides
Notifications	Sonner	—	Toasts / notifications
Dates	date-fns	3.x	Manipulation des dates
Versioning	Git / GitHub	—	Gestion des versions

Tableau 2.6 : Stack technologique du projet

Architecture logique client-serveur

L'application est structurée en 3 couches logiques distinctes :

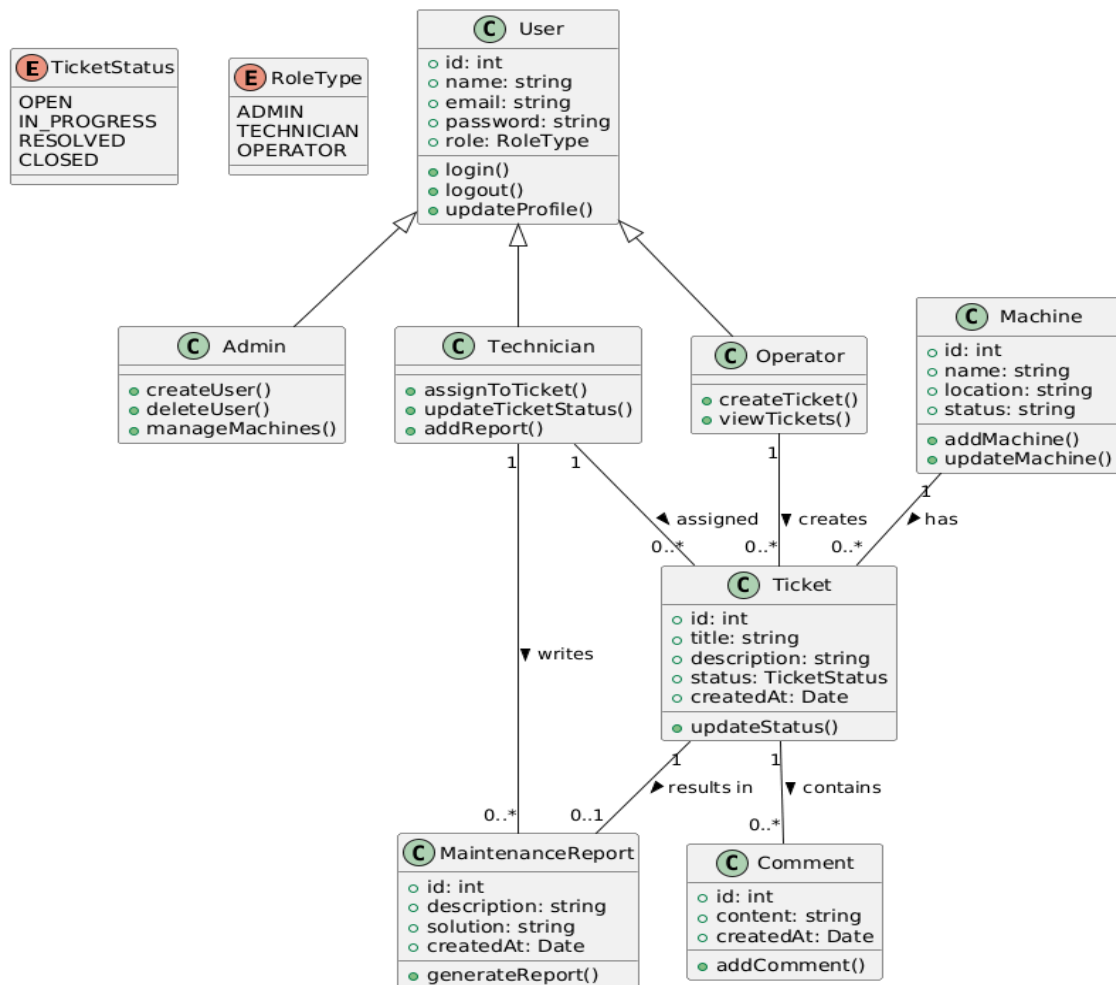
- Couche 1 : Frontend SPA React — Interface utilisateur avec React 19 + Vite 8 + Tailwind CSS 4. Le wrapper Axios dans lib/api.js gère toutes les requêtes vers le backend avec injection automatique du token Sanctum.
- Couche 2 : Backend API REST Laravel 13 — Controllers minces (TiketController, MachineController, UserController, AuthController), Middleware de rôles, Eloquent ORM pour l'accès aux données, routes/api.php pour tous les endpoints.
- Couche 3 : Base de données SQLite/MySQL — 4 tables normalisées (users, machines, tickets, logs) avec Seeders pour les données de test.

VII. Diagrammes UML

Diagramme de classes

Les entités principales et leurs relations :

- User : id, name, email, password, role, is_active — méthodes : tickets(), assignedTickets(), logs()
- Machine : id, name, code, location, status — méthodes : tickets(), isActive()
- Ticket : id, title, description, priority, status — méthodes : machine(), creator(), assignee(), logs()
- Log : id, action, details — méthodes : user(), ticket()

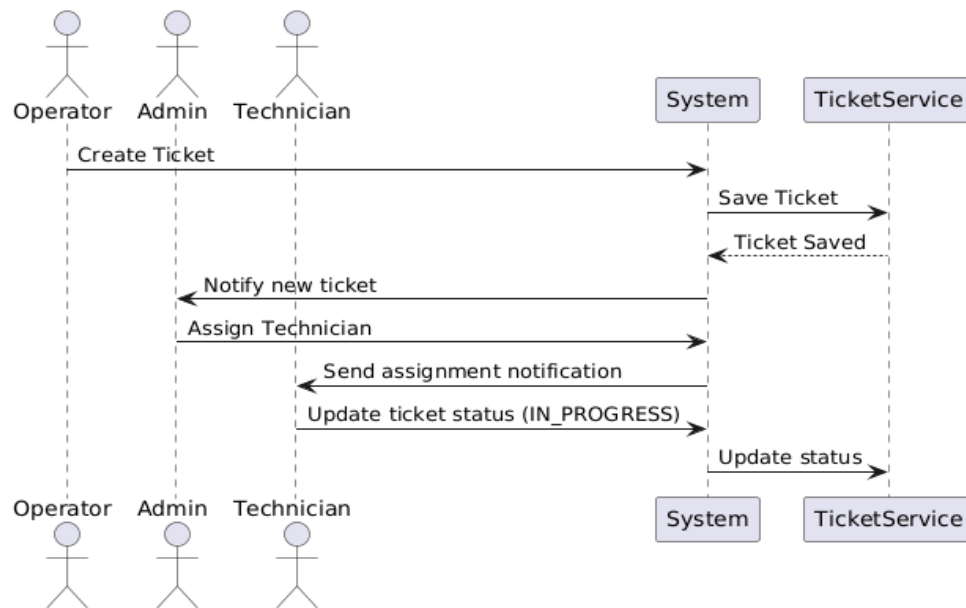


[Figure 2.4 : Diagramme de classes]

Diagramme de séquence — Création d'un ticket

Flux de séquence :

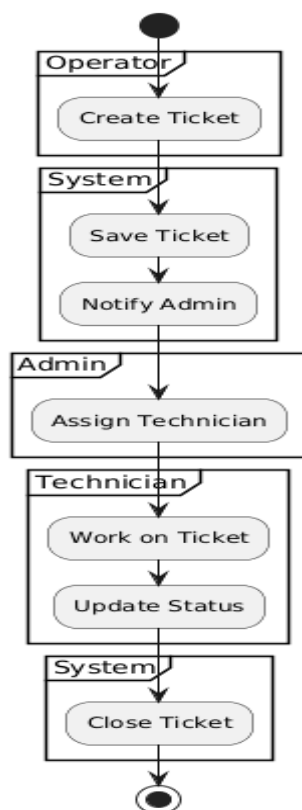
- L'opérateur scanne le QR Code de la machine via html5-qrcode.
- Le composant QRScanner déclenche une requête GET `/api/machines/{code}` pour récupérer les infos machine.
- L'opérateur remplit le formulaire TicketForm (titre, description, priorité).
- Le frontend envoie POST `/api/tickets` avec le token Sanctum en header.
- TicketController valide la requête, crée le ticket en base via Eloquent.
- Un log est créé automatiquement pour tracer l'action.
- La réponse JSON est retournée au frontend avec le ticket créé.
- Une notification toast (Sonner) confirme la création à l'utilisateur.



[Figure 2.5 : Diagramme de séquence — Création d'un ticket]

Diagramme d'état — Cycle de vie d'un ticket

- open → in_progress : Le technicien prend en charge le ticket.
- in_progress → resolved : Le technicien marque l'intervention comme résolue.
- resolved → closed : L'administrateur ferme définitivement le ticket.
- * → open : Un ticket peut être réouvert si la panne persiste.



[Figure 2.8 : Diagramme d'état — Cycle de vie d'un ticket]

CHAPITRE 3 : RÉALISATION ET MISE EN ŒUVRE

I. Les outils de travail :

Catégorie	Outil	Version	Usage
IDE	Visual Studio Code	1.87+	Édition du code source
Backend	Laravel 13 / PHP 8.4	13.x / 8.4	API REST + logique métier
Frontend	React 19 + Vite 8	19.x / 8.x	SPA + composants UI
Auth	Laravel Sanctum	4.x	Authentification par token
Base de données	SQLite / MySQL	3.x / 8.0	Persistance des données
Versioning	Git / GitHub	2.43+	Gestion des versions
Test API	Postman / Insomnia	—	Test des endpoints REST
Modélisation	Draw.io / dbdiagram.io	—	Diagrammes UML et DB
QR Code	html5-qrcode	2.x	Scan QR des machines
DB Client	TablePlus / phpMyAdmin	—	Administration base de données

Tableau 3.1 : Outils de développement utilisés

II. Environnement de développement

Environnement de développement local

Configuration matérielle :

- Processeur : Intel Core i5/i7 ou AMD Ryzen 5/7
- Mémoire RAM : 8-16 GB DDR4
- Stockage : SSD 256-512 GB

Configuration logicielle :

- OS : Windows 11 / Ubuntu 22.04 LTS
- PHP : 8.4+ avec extensions pdo, mbstring, openssl, tokenizer, xml, json, bcmath
- Composer : 2.7+
- Node.js : 20+ LTS avec npm 10+
- SQLite : 3.x (développement) / MySQL 8.0 (production)
- Git : 2.43+

Installation et configuration initiale

1	git clone https://github.com/[username]/fixflow.git && cd fixflow
2	cd fixflow-backend && composer install
3	cp .env.example .env && php artisan key:generate
4	php artisan migrate --seed
5	php artisan serve (API accessible sur http://localhost:8000)
6	cd ../client && npm install
7	npm run dev (Frontend accessible sur http://localhost:5173)

III. Structure du projet

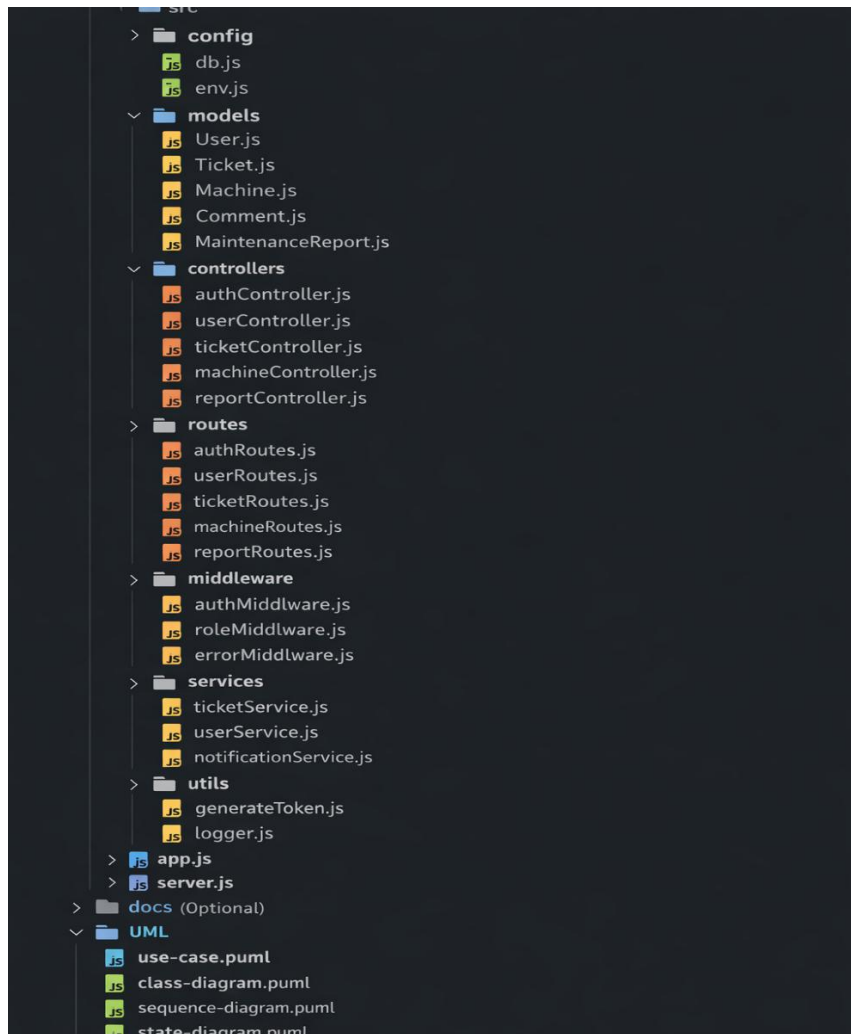
Arborescence complète du projet

fixflow-backend/

- `app/Http/Controllers/` → AuthController, TicketController, MachineController, UserController
- `app/Models/` → User, Ticket, Machine, Log
- `routes/api.php` → Toutes les routes API avec middlewares
- `database/migrations/` → Schéma versionné de la base de données
- `database/seeder/` → Données de test (UsersSeeder, MachinesSeeder, TicketsSeeder)
- `config/cors.php` → Configuration CORS pour le frontend React

client/src/

- `pages/worker/` → Interfaces opérateur (signalement pannes, liste tickets)
- `pages/technician/` → Interfaces technicien (tickets assignés, interventions)
- `pages/admin/` → Dashboard KPI, gestion users, machines, logs
- `components/` → 72+ composants réutilisables (TicketForm, QRScanner, MachineCard...)
- `context/` → AuthContext, TicketContext, MachineContext, LanguageContext
- `layouts/` → WorkerLayout, TechnicianLayout, AdminLayout
- `lib/api.js` → Wrapper Axios avec injection automatique du token
- `lib/translations.js` → Internationalisation (FR/AR/EN)
- `lib/utills.js` → Fonctions utilitaires partagées



[Figure 3.10 : Structure du projet FixFlow]

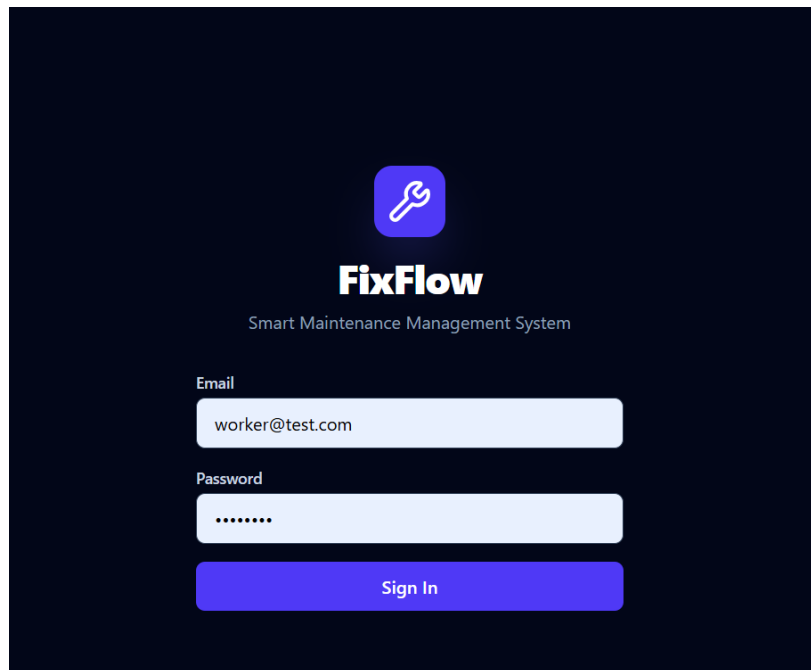
IV. Les principales interfaces

Interface de connexion (Login)

La page de connexion est le point d'entrée unique de l'application. Elle présente :

- Le logo FixFlow en en-tête
- Un formulaire avec deux champs : email et mot de passe
- La validation en temps réel avec messages d'erreur
- Redirection automatique selon le rôle (admin → /admin/dashboard, technicien → /technician, opérateur → /worker)

Techniquement, l'authentification est gérée par Laravel Sanctum. Le token retourné par l'API est stocké dans le contexte React (AuthContext) et injecté automatiquement dans chaque requête Axios via lib/api.js.

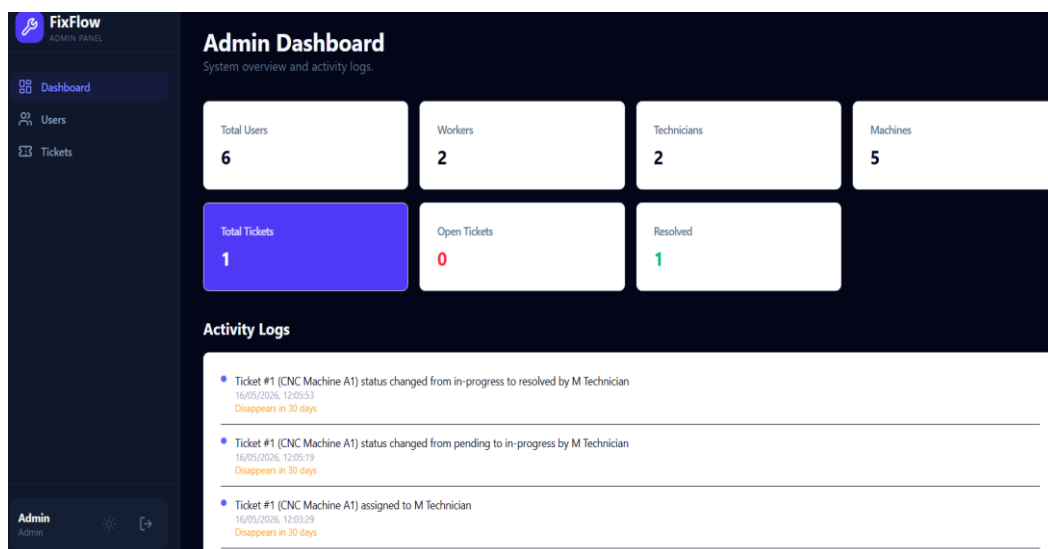


[Figure 3.1 : Interface de connexion (Login)]

Dashboard administrateur avec KPI

Le dashboard admin présente les indicateurs clés de performance :

- Nombre total de tickets créés
- Nombre de tickets ouverts (non pris en charge)
- Nombre de tickets en cours
- Nombre de tickets résolus et taux de résolution (%)
- Liste des machines les plus défaillantes
- Liste des interventions récentes

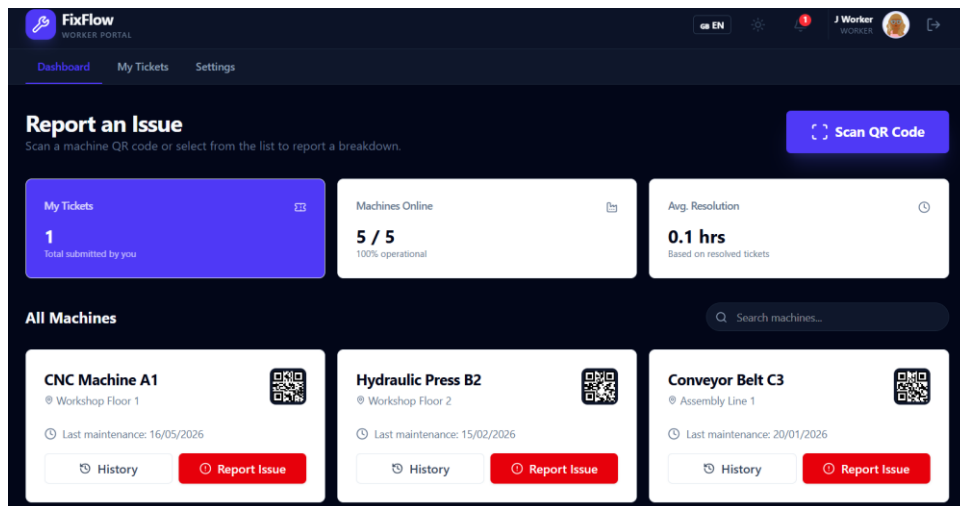


[Figure 3.2 : Dashboard administrateur avec KPI]

Interface opérateur — Signalement de panne

L'espace Worker permet à l'opérateur de :

- Lancer le scan QR pour identifier la machine automatiquement via la caméra.
- Remplir le formulaire de ticket (titre, description, priorité : low/medium/high/urgent).
- Consulter la liste de ses propres tickets avec leur statut actuel.
- Recevoir une confirmation visuelle (toast Sonner) après chaque action.

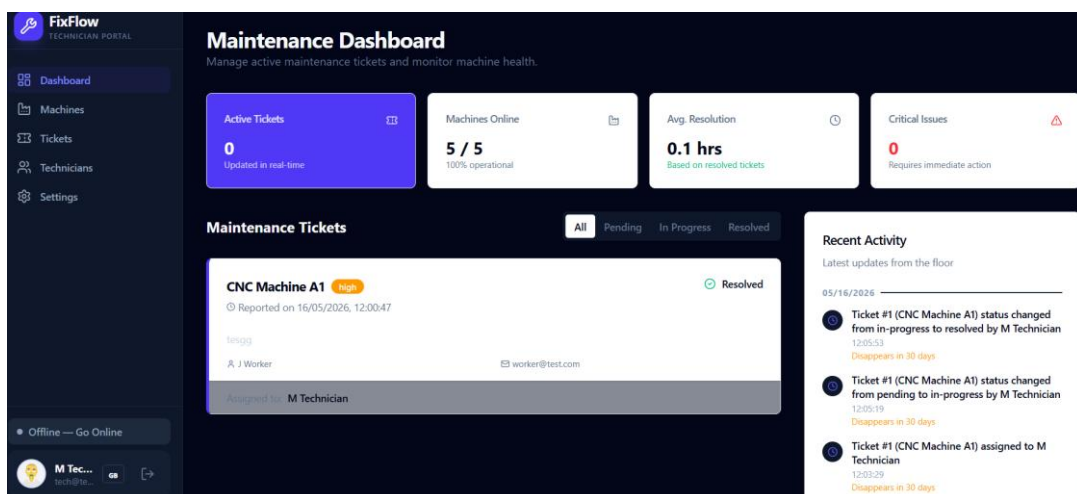


[Figure 3.3 : Interface opérateur — Signalement de panne]

Interface technicien — Gestion des tickets

L'espace Technicien affiche :

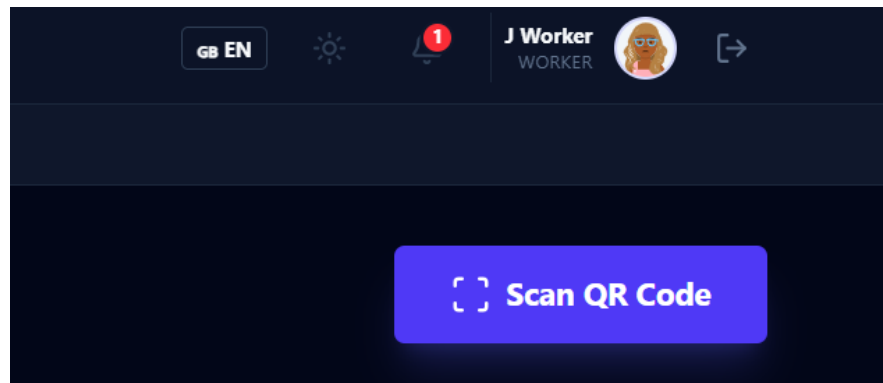
- La liste des tickets assignés au technicien connecté.
- La liste des tickets non assignés disponibles à prendre en charge.
- Pour chaque ticket : machine, priorité, description, date de création.
- Boutons d'action : Prendre en charge, Marquer résolu.



[Figure 3.4 : Interface technicien — Gestion des tickets]

Scan QR Code machine

Le composant QRScanner utilise la bibliothèque html5-qrcode pour accéder à la caméra du navigateur et décoder le QR Code des machines. Une fois le code détecté, une requête GET /api/machines/{code} est envoyée automatiquement pour récupérer les informations de la machine et pré-remplir le formulaire de ticket.



[Figure 3.5 : Scan QR Code machine]

V. Tests et validation

Tests unitaires et fonctionnels

La qualité du code a été garantie à travers une stratégie de tests :

- Tests API avec PHPUnit : AuthController (login, logout, register), TicketController (CRUD complet), MachineController (CRUD + QR), UserController (gestion rôles).
- Tests de routes API avec Postman : Vérification de chaque endpoint avec et sans token, avec différents rôles.
- Tests d'interface : Navigation entre les espaces, création de tickets, scan QR (simulation), filtres.

Tests de performance

- Dashboard admin : < 2 s
- Création de ticket : < 0.5 s
- Scan QR + récupération machine : < 1 s
- Liste des tickets (100 items) : < 0.8 s

VI. Déploiement

Environnement de production

- OS : Ubuntu Server 22.04 LTS
- Web Server : Nginx 1.24 avec configuration optimisée Laravel
- PHP : 8.4 FPM avec OPcache activé
- Base de données : MySQL 8.0
- SSL : Certificat Let's Encrypt avec renouvellement automatique

Procédure de déploiement

- php artisan down — Mise en mode maintenance
- git pull origin main — Mise à jour du code
- composer install --no-dev --optimize-autoloader
- php artisan migrate --force
- php artisan config:cache && php artisan route:cache

- `cd client && npm ci && npm run build`
- `php artisan up` — Remise en ligne

Checklist de mise en production

Étape	Action	Statut
1	Variables .env production configurées (APP_DEBUG=false)	✓ Validé
2	Base de données migrée (php artisan migrate --force)	✓ Validé
3	Frontend compilé (npm run build)	✓ Validé
4	Clés Sanctum configurées et testées	✓ Validé
5	CORS configuré pour le domaine de production	✓ Validé
6	Rôles et utilisateurs de base créés (Seeders)	✓ Validé
7	HTTPS + certificat SSL actif	✓ Validé
8	Tests de fumée passés sur l'API	✓ Validé
9	Accès vérifié pour chaque profil (admin, tech, worker)	✓ Validé
10	Sauvegarde automatique de la base configurée	✓ Validé

Tableau 3.5 : Checklist de déploiement en production



CONCLUSION

Ce stage de fin de formation au sein de Nouvelle Aveiro Maroc m'a permis de concevoir et développer FixFlow, une application web complète de gestion de maintenance industrielle répondant à un besoin réel et critique de l'entreprise. Ce projet représente une expérience professionnelle et humaine particulièrement enrichissante.

Synthèse du travail réalisé

Au cours de ces quatre semaines de stage, j'ai mené à bien l'ensemble des missions qui m'avaient été confiées :

- Conception architecturale : Architecture client-serveur API REST (Laravel 13) + SPA (React 19) avec modélisation UML complète.
- Développement backend : API REST complète avec 4 contrôleurs, authentification Sanctum, 4 tables de base de données avec migrations et seeders, système de logs.
- Développement frontend : SPA React avec 3 espaces utilisateurs (Worker, Technicien, Admin), 72+ composants UI, intégration QR Code, contexts React, routage protégé par rôle.
- Qualité et tests : Tests API Postman, tests PHPUnit, validation des performances.
- Documentation : Documentation technique (README, guide API), documentation utilisateur par profil, rapport de recette.
- Déploiement : Préparation de l'environnement de production avec Nginx, PHP-FPM, MySQL, SSL.

Apport personnel et professionnel

Sur le plan technique, j'ai approfondi ma maîtrise de Laravel 13 et de l'écosystème PHP moderne (Sanctum, Eloquent, API Resources). J'ai renforcé mes compétences React 19 avec les hooks avancés, les contextes et le routage protégé. L'intégration de html5-qrcode m'a ouvert à de nouvelles possibilités de développement orienté hardware.

Sur le plan personnel, ce stage m'a permis de découvrir le fonctionnement d'une entreprise industrielle. L'accueil chaleureux de l'équipe Nouvelle Aveiro Maroc et la disponibilité de mon tuteur M. Achraf Karmaoui ont rendu cette expérience particulièrement enrichissante et motivante.

Perspectives d'évolution

- Application mobile React Native pour le scan QR et la gestion des tickets en mobilité.
- Notifications push en temps réel via WebSockets (Laravel Echo + Pusher) pour les nouveaux tickets.
- Maintenance prédictive par analyse des historiques de pannes avec Machine Learning.
- Module de planification de la maintenance préventive avec calendrier.
- Tableaux de bord avancés avec graphiques (Recharts ou Chart.js).
- Internationalisation complète (FR/AR/EN) via le LanguageContext déjà en place.

En conclusion, ce stage de fin de formation a été une réussite tant sur le plan professionnel que personnel. Il m'a permis de consolider les compétences acquises à l'ISTA Aït Melloul et de contribuer concrètement à l'amélioration des processus de Nouvelle Aveiro Maroc.



RÉFÉRENCES

- [1] Laravel Documentation, « Laravel 13 - The PHP Framework For Web Artisans », <https://laravel.com/docs/13.x>, Consulté le 15 avril 2026.
- [2] Laravel Sanctum, « API Token Authentication », <https://laravel.com/docs/13.x/sanctum>, Consulté le 10 avril 2026.
- [3] React Documentation, « React — The library for web and native user interfaces », <https://react.dev>, Consulté le 12 avril 2026.
- [4] Vite Documentation, « Vite — Next Generation Frontend Tooling », <https://vitejs.dev/guide>, Consulté le 08 avril 2026.
- [5] Tailwind CSS Documentation, « Tailwind CSS v4 », <https://tailwindcss.com/docs>, Consulté le 14 avril 2026.
- [6] React Router, « React Router DOM v7 », <https://reactrouter.com/en/main>, Consulté le 11 avril 2026.
- [7] html5-qrcode, « HTML5 QR Code scanner », <https://github.com/mebjas/html5-qrcode>, Consulté le 13 avril 2026.
- [8] Axios Documentation, « Axios HTTP Client », <https://axios-http.com/docs/intro>, Consulté le 09 avril 2026.
- [9] PHPUnit Documentation, « PHPUnit - The PHP Testing Framework », <https://phpunit.de/documentation.html>, Consulté le 16 avril 2026.
- [10] OFPPT, « Référentiel de formation Développement Digital », ISTA Aït Melloul, 2024-2026.
- [11] Mdn Web Docs, « HTTP — HyperText Transfer Protocol », <https://developer.mozilla.org/en-US/docs/Web/HTTP>, Consulté le 07 avril 2026.
- [12] date-fns, « Modern JavaScript date utility library », <https://date-fns.org/docs/Getting-Started>, Consulté le 10 avril 2026.